

Engineering



& Physics

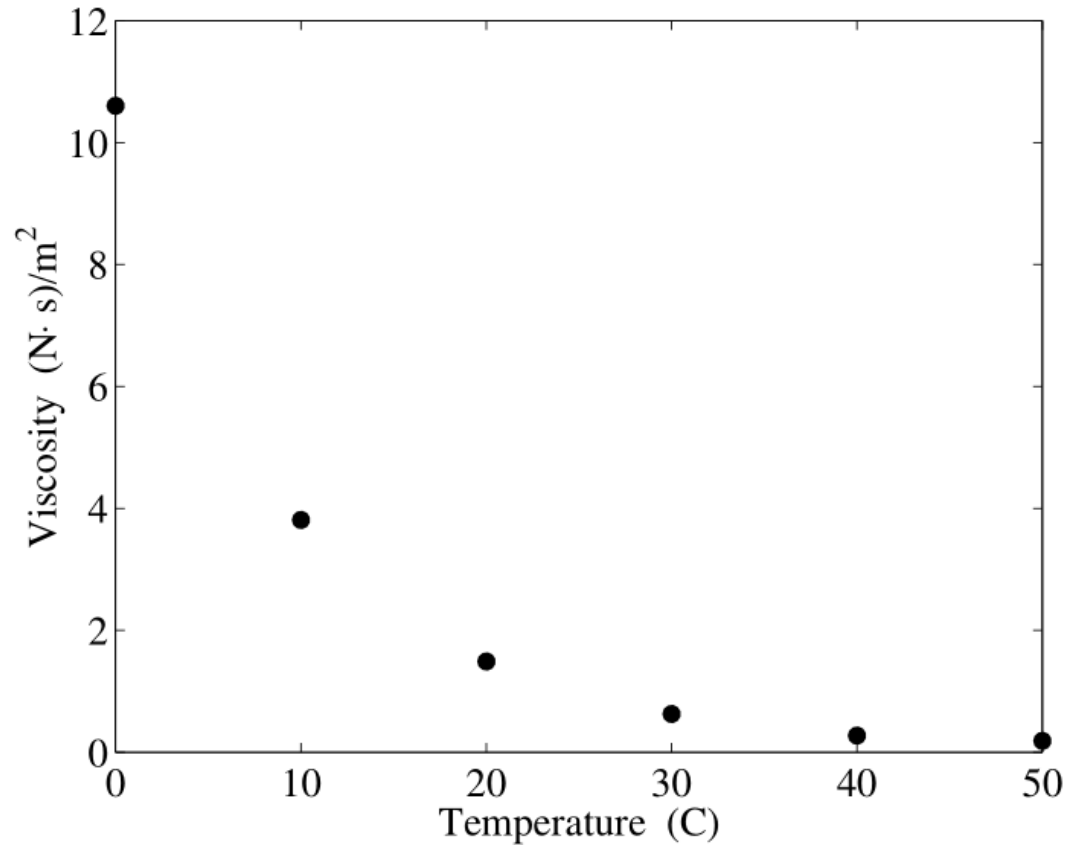
PHYS 351

Interpolation and Fitting

Dr. Daugherty

Abilene Christian University

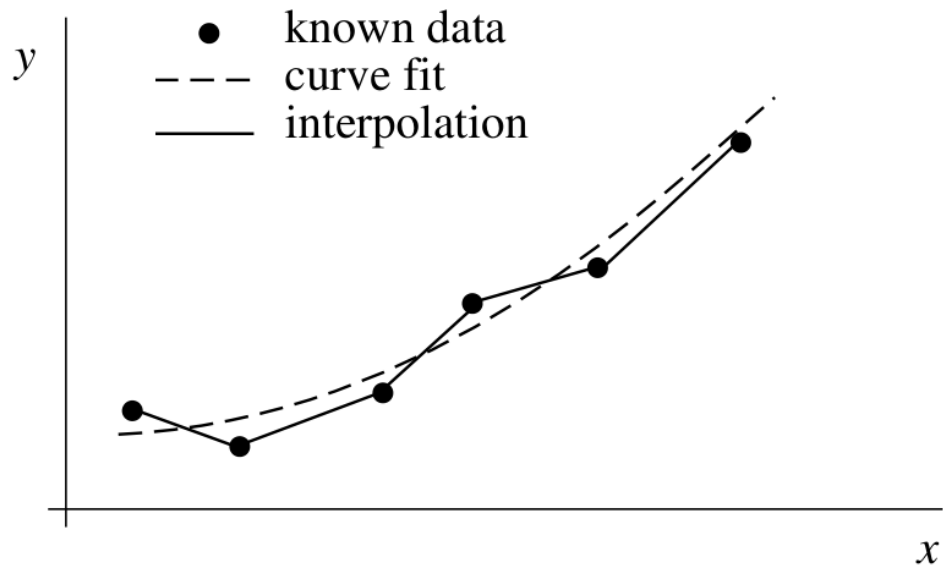
Problem



Make a good guess about what happens in between the data points.

What is V at $T=25$ C?

Approaches



Interpolation: connect the dots with piecewise equation

Fitting: find a single function which approximates the data

IMPORTANT – Interpolation goes through every point, while fitting will “average between” them.

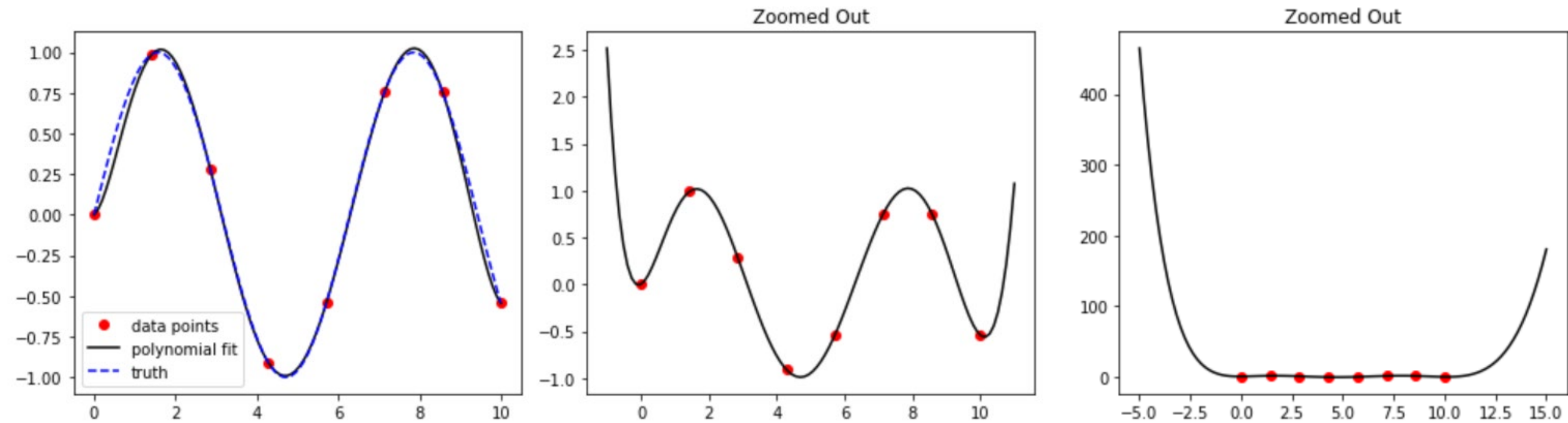
Approaches

	GOOD	BAD
Interpolation	• fast and easy (connect-the-dots) • standard approaches that nearly always work	• does not reduce noise • does not reduce size of data
Fitting	• reduces noise • data reduction • easy to evaluate after the fit	• we usually don't know what function to use • fitting parameters can be extremely hard

What Doesn't Work

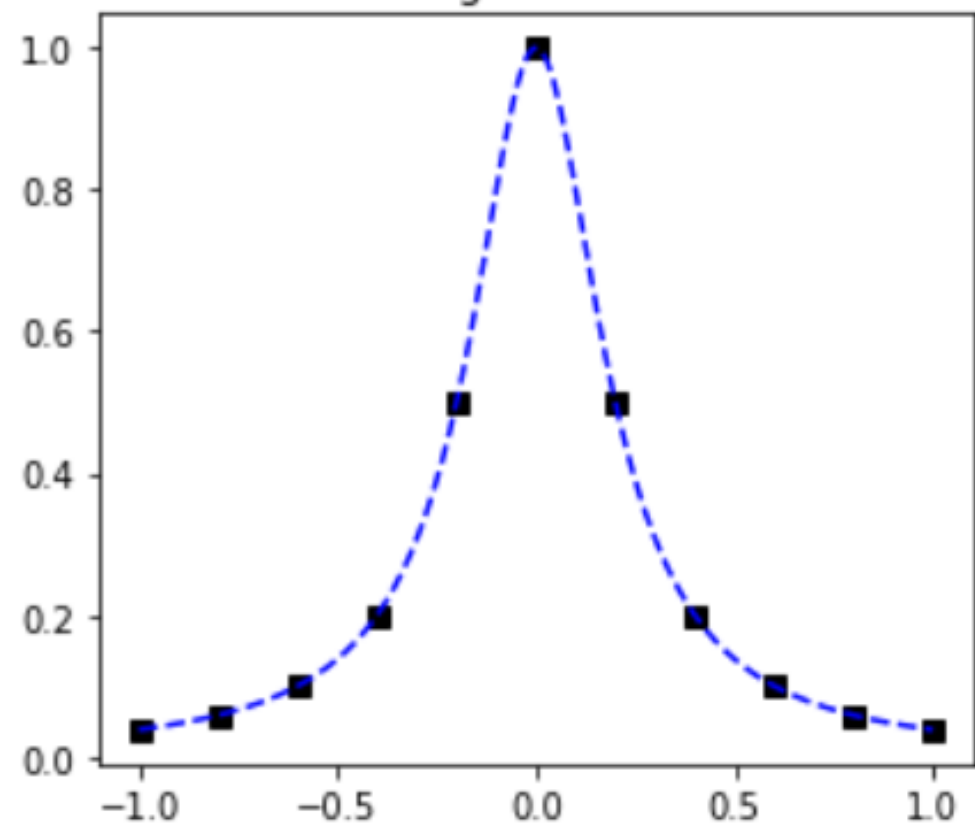
THEOREM: For any set of n points, there is one unique polynomial of degree $n-1$ that goes through them.

So why not just do that?

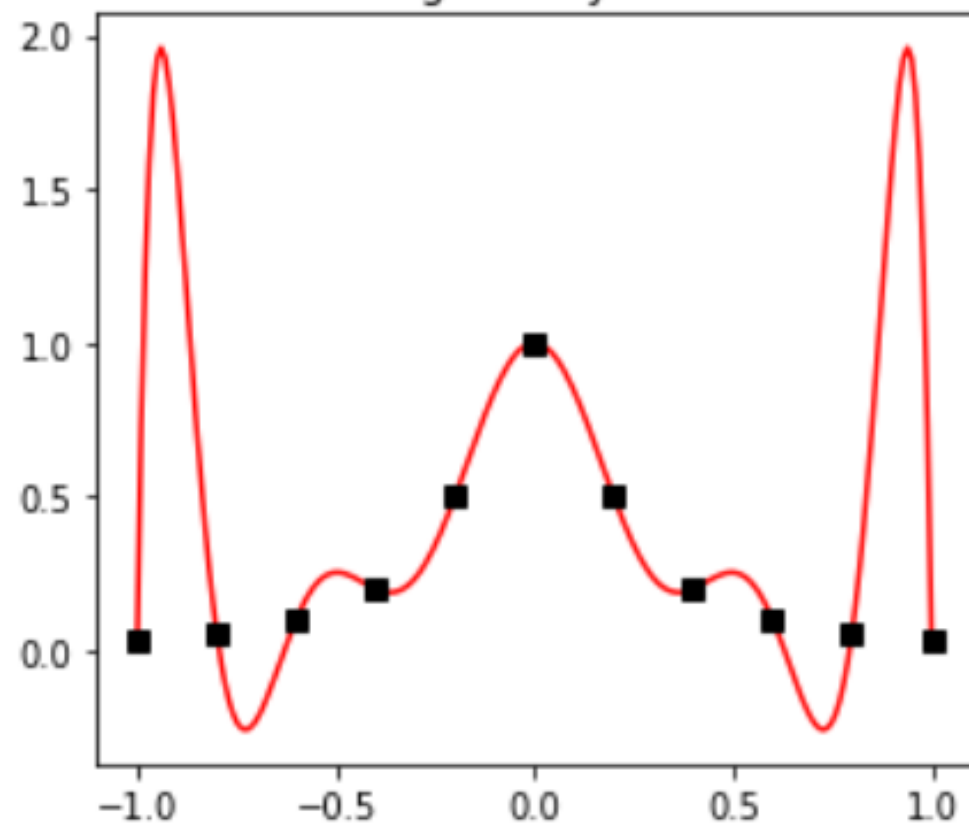


EXTRAPOLATION – what happens beyond the data points?

Runge Function



10th Degree Polynomial Fit



Fitting

Least Squares

https://phet.colorado.edu/sims/html/curve-fitting/latest/curve-fitting_en.html

- mathematical definition of “best” fit
- a **solved** problem for any polynomial

numpy.polyfit

`numpy.polyfit(x, y, deg, rcond=None, full=False, w=None, cov=False)`

Least squares polynomial fit.

numpy.polyval

`numpy.polyval(p, x)`

Evaluate a polynomial at specific values.

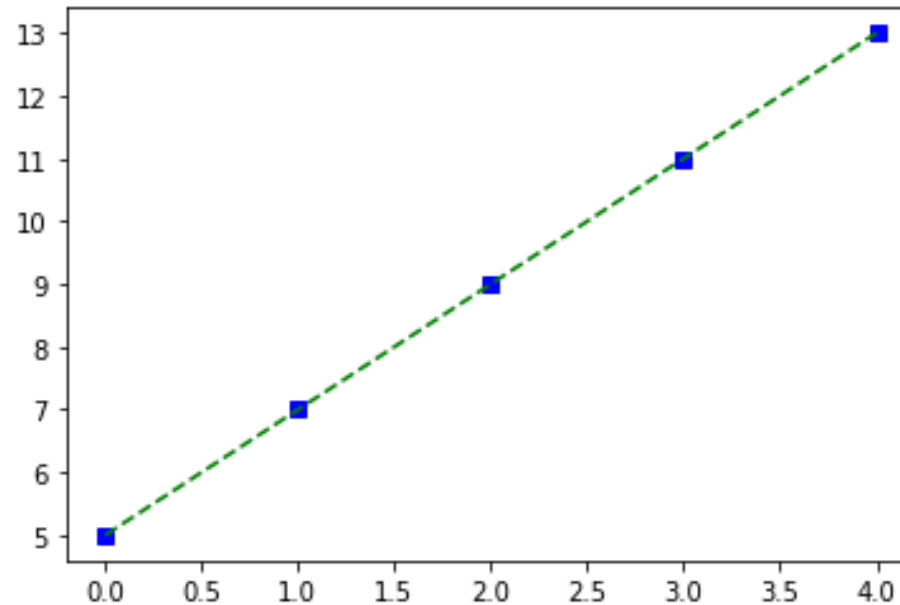
- For simple polynomial fits use `polyfit`
- Use the fit results in `polyval`

```
xdata = np.arange(0,5)
m = 2
b = 5
ydata = m*xdata + b
p = np.polyfit(xdata,ydata,1)
print('Actual:\t',m,b)
print('Fit:\t',p)

x = np.linspace(0,4)
y = np.polyval(p, x)

plt.plot(xdata,ydata,'bs')
plt.plot(x,y,'g--')
plt.show()
```

Actual: 2 5
Fit: [2. 5.]



Local Optimization

For an arbitrary fit function $f(x)$ we need some tools:

- define a measure-of-badness a.k.a Loss Function like χ^2
- pick a starting point with initial guesses for parameters
- gradient descent: adjust each parameter to minimize loss

Gradient Descent is (perhaps) the most important algorithm of last few decades. Modern neural networks do this with $\sim 10^{11}$ parameters

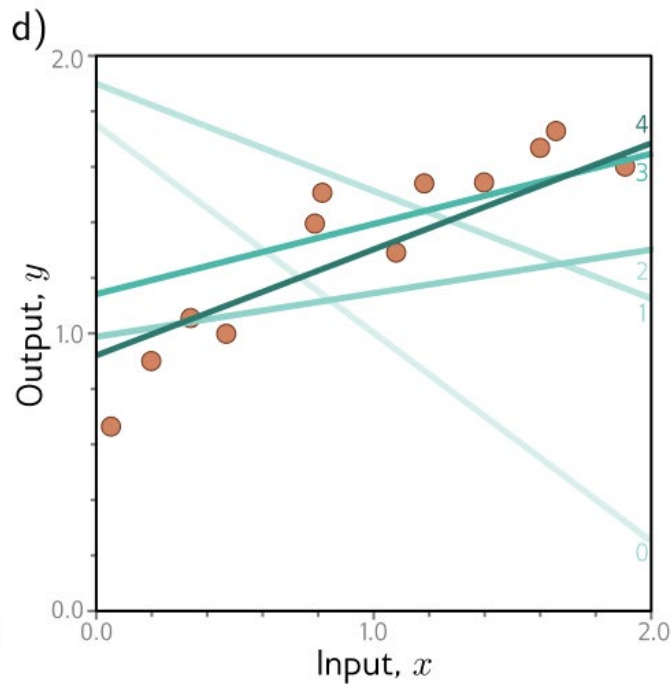
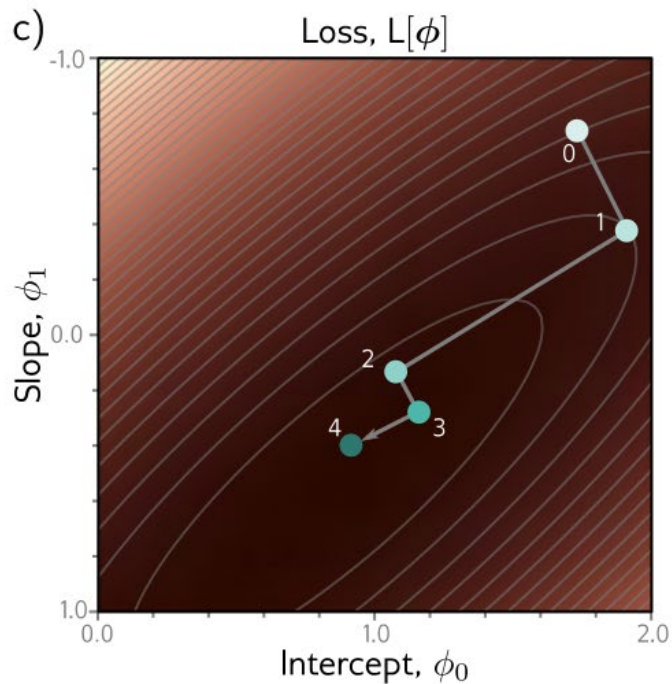
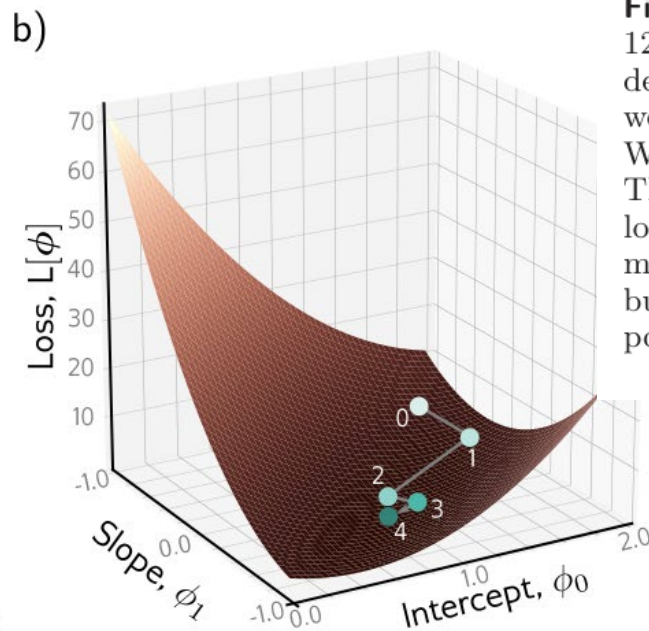
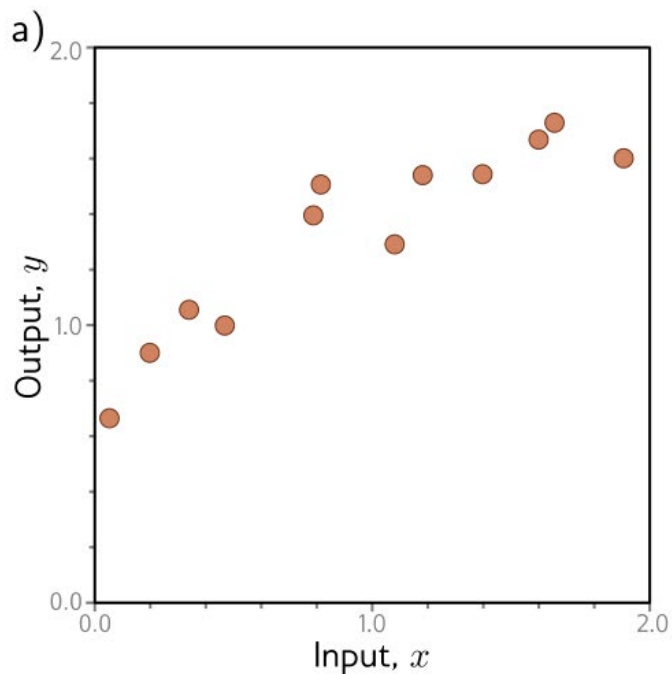


Figure 6.1 Gradient descent for the linear regression model. a) Training set of $I = 12$ input/output pairs $\{x_i, y_i\}$. b) Loss function showing iterations of gradient descent. We start at point 0 and move in the steepest downhill direction until we can improve no further to arrive at point 1. We then repeat this procedure. We measure the gradient at point 1 and move downhill to point 2 and so on. c) This can be visualized better as a heatmap, where the brightness represents the loss. After only four iterations, we are already close to the minimum. d) The model with the parameters at point 0 (lightest line) describes the data very badly, but each successive iteration improves the fit. The model with the parameters at point 4 (darkest line) is already a reasonable description of the training data.

Consider applying gradient descent to the 1D linear regression model from chapter 2. The model $f[x, \phi]$ maps a scalar input x to a scalar output y and has parameters $\phi = [\phi_0, \phi_1]^T$, which represent the y-intercept and the slope:

$$\begin{aligned} y &= f[x, \phi] \\ &= \phi_0 + \phi_1 x. \end{aligned} \quad (6.4)$$

Given a dataset $\{x_i, y_i\}$ containing I input/output pairs, we choose the least squares loss function:

$$\begin{aligned} L[\phi] &= \sum_{i=1}^I \ell_i = \sum_{i=1}^I (f[x_i, \phi] - y_i)^2 \\ &= \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2, \end{aligned} \quad (6.5)$$

Use `curve_fit` to fit data to an arbitrary function

Be careful! It will only find a **local** minima from the initial guess

```
scipy.optimize.least_squares(fun, x0, jac='2-point', bounds=(- inf, inf), method='trf',  
ftol=1e-08, xtol=1e-08, gtol=1e-08, x_scale=1.0, loss='linear', f_scale=1.0, diff_step=None,  
tr_solver=None, tr_options={}, jac_sparsity=None, max_nfev=None, verbose=0, args=(),  
kwargs={})
```

[source]

Solve a nonlinear least-squares problem with bounds on the variables.

Given the residuals $f(x)$ (an m -D real function of n real variables) and the loss function $\rho(s)$ (a scalar function), `least_squares` finds a local minimum of the cost function $F(x)$:

```
minimize F(x) = 0.5 * sum(rho(f_i(x)**2), i = 0, ..., m - 1)  
subject to lb <= x <= ub
```

The purpose of the loss function $\rho(s)$ is to reduce the influence of outliers on the solution.

Parameters: `fun` : callable

Function which computes the vector of residuals, with the signature `fun(x, *args, **kwargs)`, i.e., the minimization proceeds with respect to its first argument. The argument `x` passed to this function is an ndarray of shape $(n,)$ (never a scalar, even for $n=1$). It must allocate and return a 1-D array_like of shape $(m,)$ or a scalar. If the argument `x` is complex or the function `fun` returns complex residuals, it must be wrapped in a real function of real arguments, as shown at the end of the Examples section.

```
scipy.optimize.curve_fit(f, xdata, ydata, p0=None, sigma=None, absolute_sigma=False,  
check_finite=True, bounds=(- inf, inf), method=None, jac=None, *, full_output=False,  
**kwargs)
```

[source]

Use non-linear least squares to fit a function, `f`, to data.

Assumes `ydata = f(xdata, *params) + eps`.

Parameters: `f` : callable

The model function, $f(x, \dots)$. It must take the independent variable as the first argument and the parameters to fit as separate remaining arguments.

xdata : array_like or object

The independent variable where the data is measured. Should usually be an M -length sequence or an (k,M) -shaped array for functions with k predictors, but can actually be any object.

ydata : array_like

The dependent data, a length M array - nominally `f(xdata, ...)`.

p0 : array_like, optional

Initial guess for the parameters (length N). If `None`, then the initial values will all be 1 (if the number of parameters for the function can be determined using introspection, otherwise a `ValueError` is raised).

sigma : None or M -length sequence or $M \times M$ array, optional

Determines the uncertainty in `ydata`. If we define residuals as `r = ydata - f(xdata, *popt)`, then the interpretation of `sigma` depends on its number of dimensions:

- A 1-D `sigma` should contain values of standard deviations of errors in `ydata`. In this case, the optimized function is `chisq = sum((r / sigma) ** 2)`.
- A 2-D `sigma` should contain the covariance matrix of errors in `ydata`. In this case, the optimized function is `chisq = r.T @ inv(sigma) @ r`.

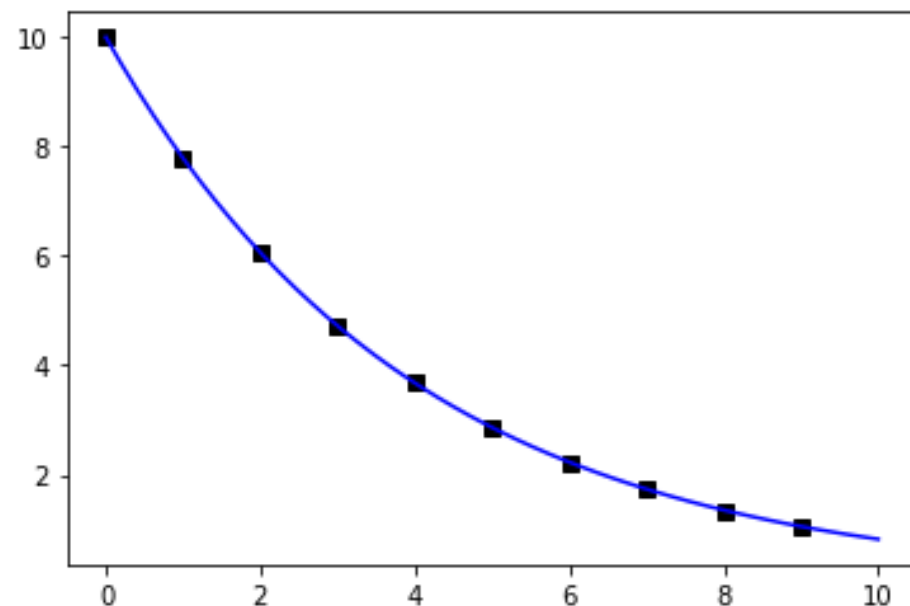
```
scipy.optimize.curve_fit(f, xdata, ydata, p0=None, sigma=None, absolute_sigma=False,
check_finite=True, bounds=(- inf, inf), method=None, jac=None, *, full_output=False,
**kwargs)
```

[source]

```
1 from scipy.optimize import curve_fit
```

```
1 def fitfun(x,a,b):
2 |     return a*np.exp(b*x)
3
4 popt, pcov = curve_fit(fitfun,xdata,ydata)
5 print(popt)
6
7 xfit = np.linspace(0,NUM)
8 yfit = fitfun(xfit, *popt) # *popt unpacks array into function args
9
10 plt.plot(xdata,ydata,'ks')
11 plt.plot(xfit,yfit,'b-')
```

```
[10.00313449 -0.25024117]
[<matplotlib.lines.Line2D at 0x7fd78ca56650>]
```



```
1 NUM = 10
2 xdata = np.arange(0,NUM)
3 ydata = 10*np.exp(-xdata*0.25) + 0.005*np.random.randn(NUM)
```

curve_fit WARNING

- If it gets a terrible fit it will not return any warning
- Plot the result
- Look at pcov for HUGE errors!



Var_1	$\text{Cov}_{1,2}$	$\text{Cov}_{1,3}$
$\text{Cov}_{1,2}$	Var_2	$\text{Cov}_{2,3}$
$\text{Cov}_{1,3}$	$\text{Cov}_{2,3}$	Var_3

Advanced Fitting

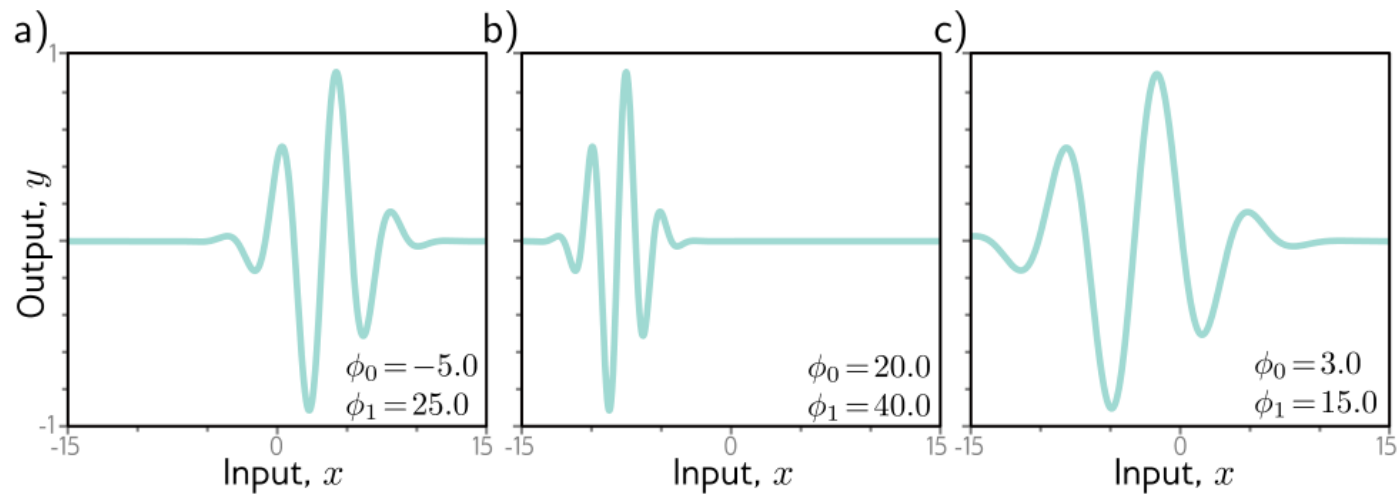


Figure 6.2 Gabor model. This nonlinear model maps scalar input x to scalar output y and has parameters $\phi = [\phi_0, \phi_1]^T$. It describes a sinusoidal function that decreases in amplitude with distance from its center. Parameter $\phi_0 \in \mathbb{R}$ determines the position of the center. As ϕ_0 increases, the function moves left. Parameter $\phi_1 \in \mathbb{R}^+$ squeezes the function along the x -axis relative to the center. As ϕ_1 increases, the function narrows. a-c) Model with different parameters. ([Interactive figure](#))

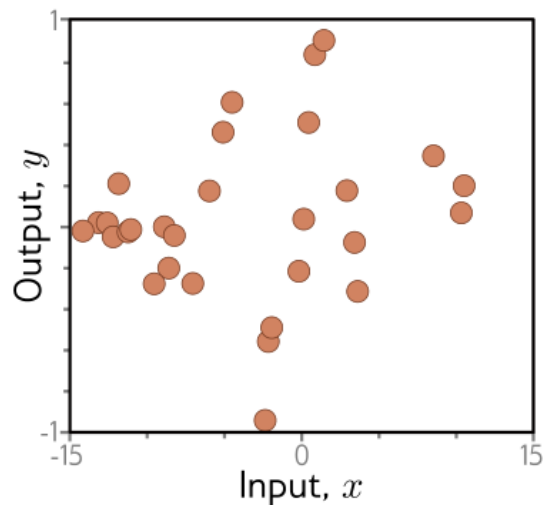


Figure 6.3 Training data for fitting the Gabor model. The training dataset contains 28 input/output examples $\{x_i, y_i\}$. These data were created by uniformly sampling $x_i \in [-15, 15]$, passing the samples through a Gabor model with parameters $\phi = [0.0, 16.6]^T$, and adding normally distributed noise.

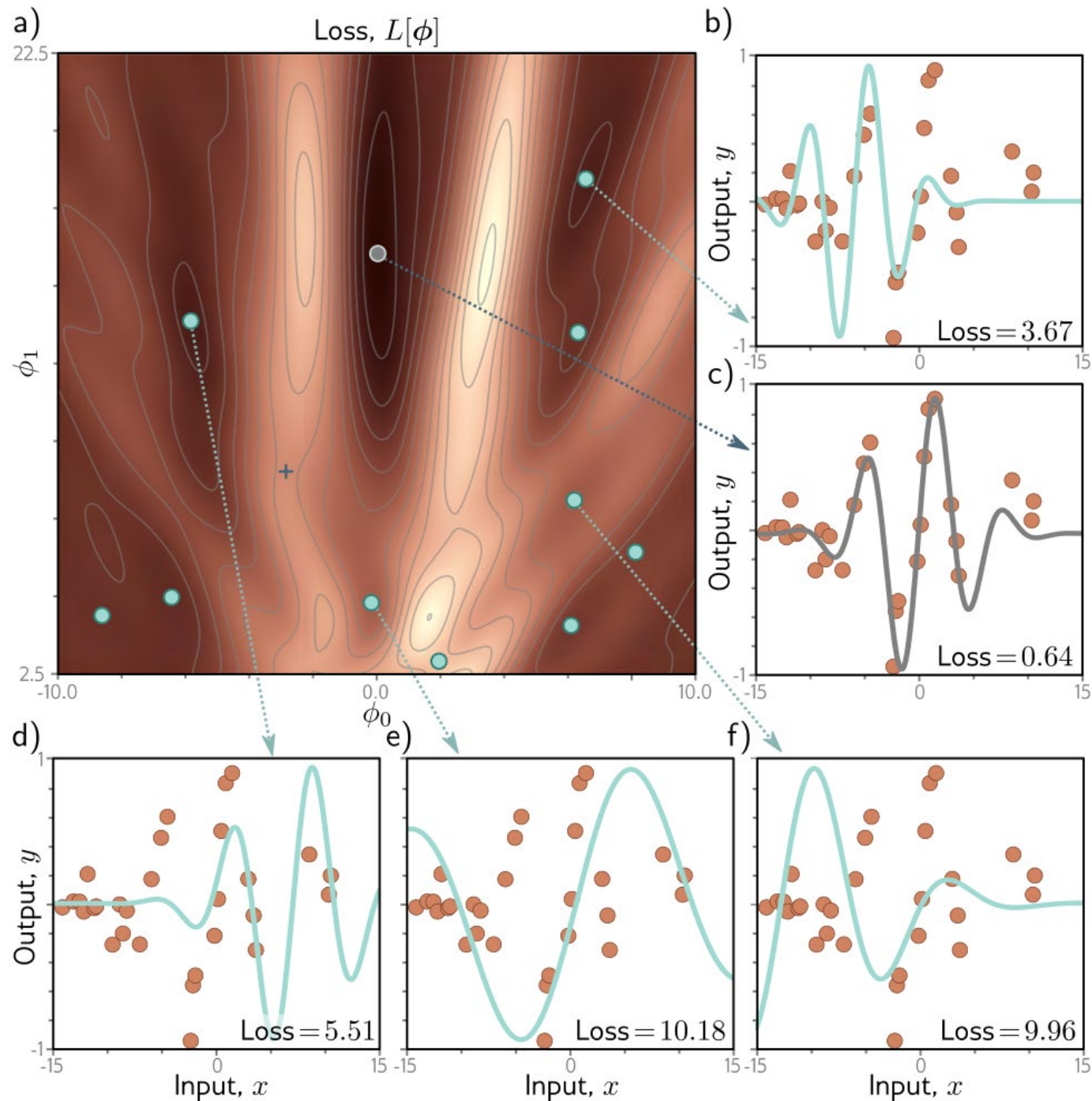


Figure 6.4 Loss function for the Gabor model. a) The loss function is non-convex, with multiple local minima (cyan circles) in addition to the global minimum (gray circle). It also contains saddle points where the gradient is locally zero, but the function increases in one direction and decreases in the other. The blue cross is an example of a saddle point; the function decreases as we move horizontally in either direction but increases as we move vertically. b–f) Models associated with the different minima. In each case, there is no small change that decreases the loss. Panel (c) shows the global minimum, which has a loss of 0.64. ([Interactive figure](https://udlbook.github.io/udlfigures/))

Global Optimization

<https://docs.scipy.org/doc/scipy/reference/optimize.html#global-optimization>

Global optimization

<code>basinhopping</code> (func, x0[, niter, T, stepsize, ...])	Find the global minimum of a function using the basin-hopping algorithm.
<code>brute</code> (func, ranges[, args, Ns, full_output, ...])	Minimize a function over a given range by brute force.
<code>differential_evolution</code> (func, bounds[, args, ...])	Finds the global minimum of a multivariate function.
<code>shgo</code> (func, bounds[, args, constraints, n, ...])	Finds the global minimum of a function using SHG optimization.
<code>dual_annealing</code> (func, bounds[, args, ...])	Find the global minimum of a function using Dual Annealing.
<code>direct</code> (func, bounds, *[, args, eps, maxfun, ...])	Finds the global minimum of a function using the DIRECT algorithm.

Notes

needs initial guess and “temperature”

give ranges, will “polish” final result

great! Needs bounds

(haven’t used this much)

(haven’t used this much)

(haven’t used this much)

curve_fit is a local optimizer from your initial guess. You can always try a global optimizer instead!

iminuit

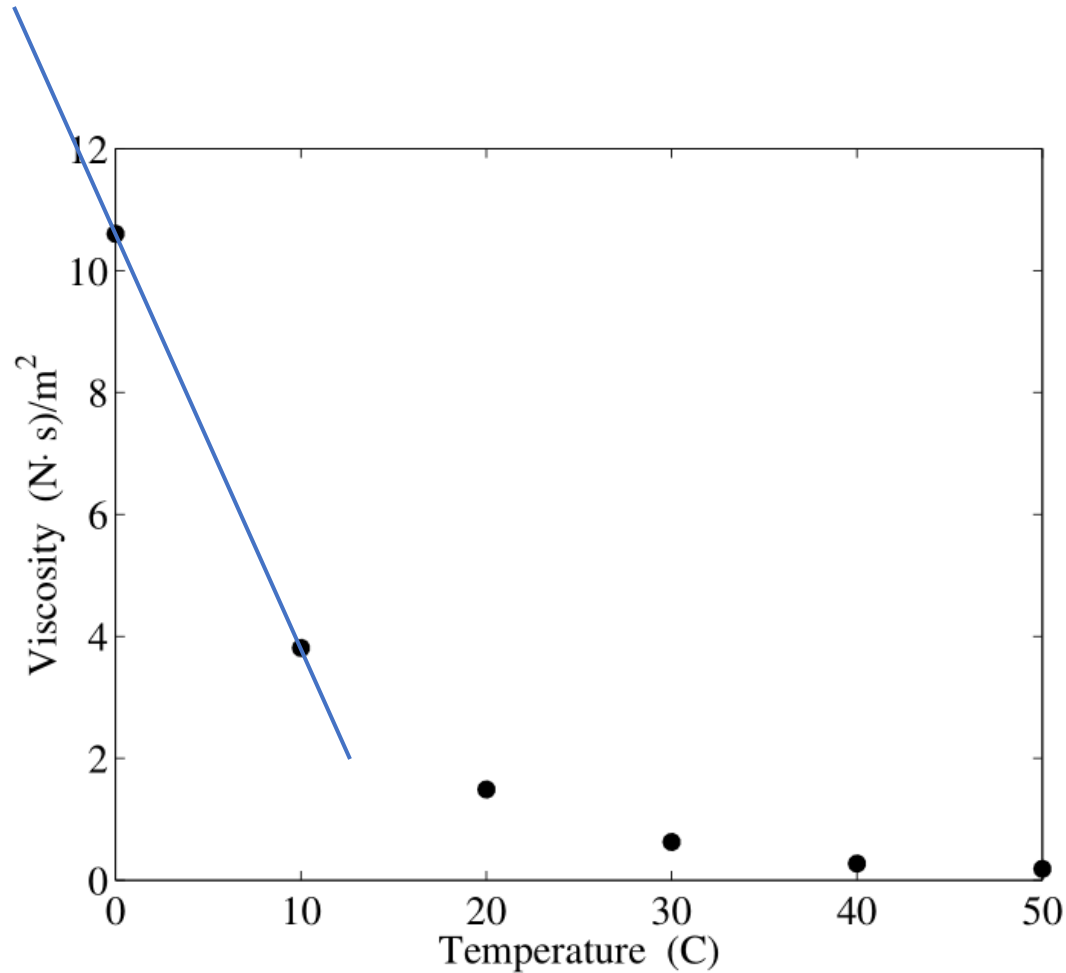
- Yes, `curve_fit` is easy to use but a little dumb
- Need an industrial-strength fitter? Try iminuit
- <https://iminuit.readthedocs.io/en/stable/index.html>

Fitting Notes

- easy for simple polynomials
- do NOT use high order polynomials
- you may need to transform your data first
 - example: if $y(x)$ is exponential, try fitting the log of y
- General purpose fitting to an arbitrary function is hard, but tools do exist...

Interpolation

Problem



Do a linear interpolation by hand

Given:

$$x_1 = 0, \quad y_1 = 10.5$$

$$x_2 = 10, \quad y_2 = 4$$

Find y at $x=5$

Linear

<https://numpy.org/doc/stable/reference/generated/numpy.interp.html>

numpy.interp

`numpy.interp(x, xp, fp, left=None, right=None, period=None)`

[\[source\]](#)

One-dimensional linear interpolation for monotonically increasing sample points.

Returns the one-dimensional piecewise linear interpolant to a function with given discrete data points (xp, fp) , evaluated at x .

Parameters:

x : *array_like*

The x-coordinates at which to evaluate the interpolated values.

xp : *1-D sequence of floats*

The x-coordinates of the data points, must be increasing if argument *period* is not specified. Otherwise, *xp* is internally sorted after normalizing the periodic boundaries with `xp = xp % period`.

fp : *1-D sequence of float or complex*

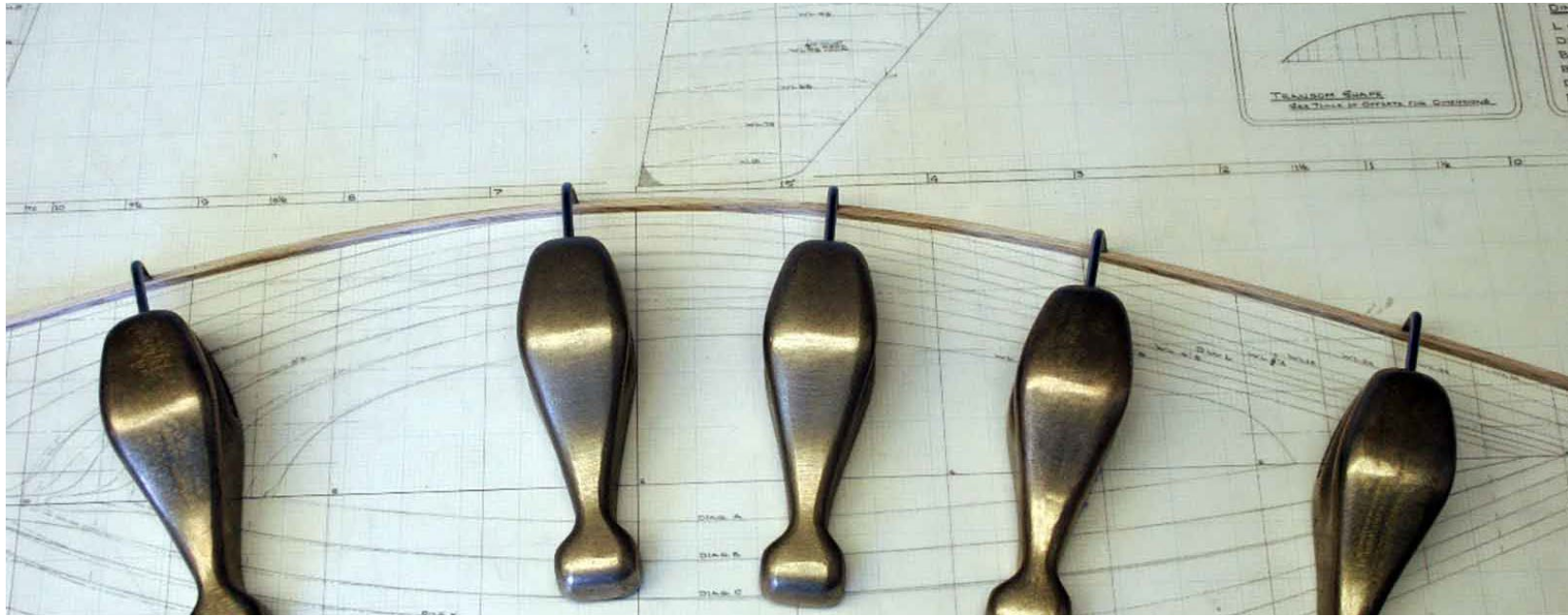
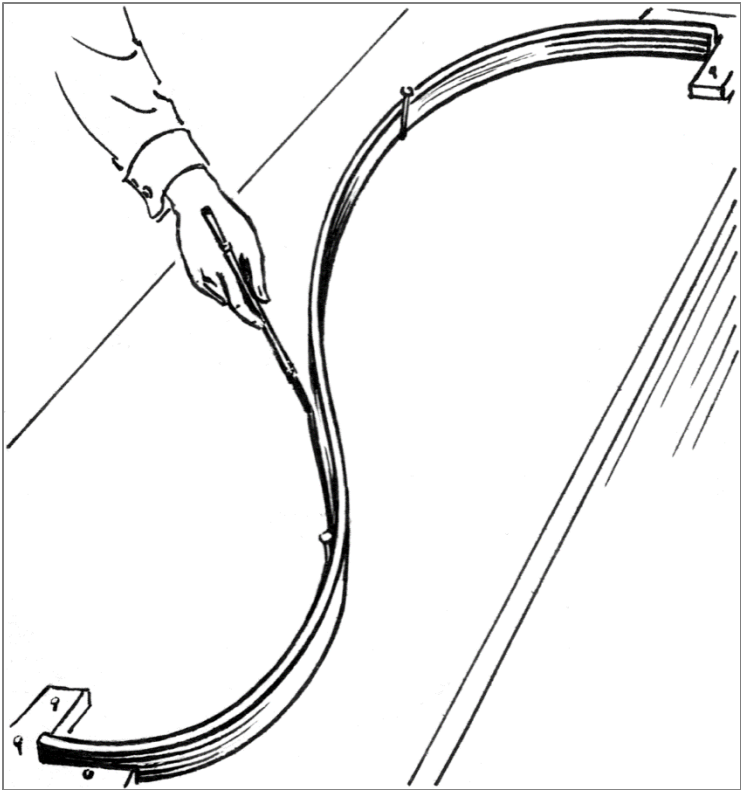
The y-coordinates of the data points, same length as *xp*.

```
1 xp = [1, 2, 3]
2 fp = [3, 2, 0]
3 np.interp(2.5, xp, fp)
```

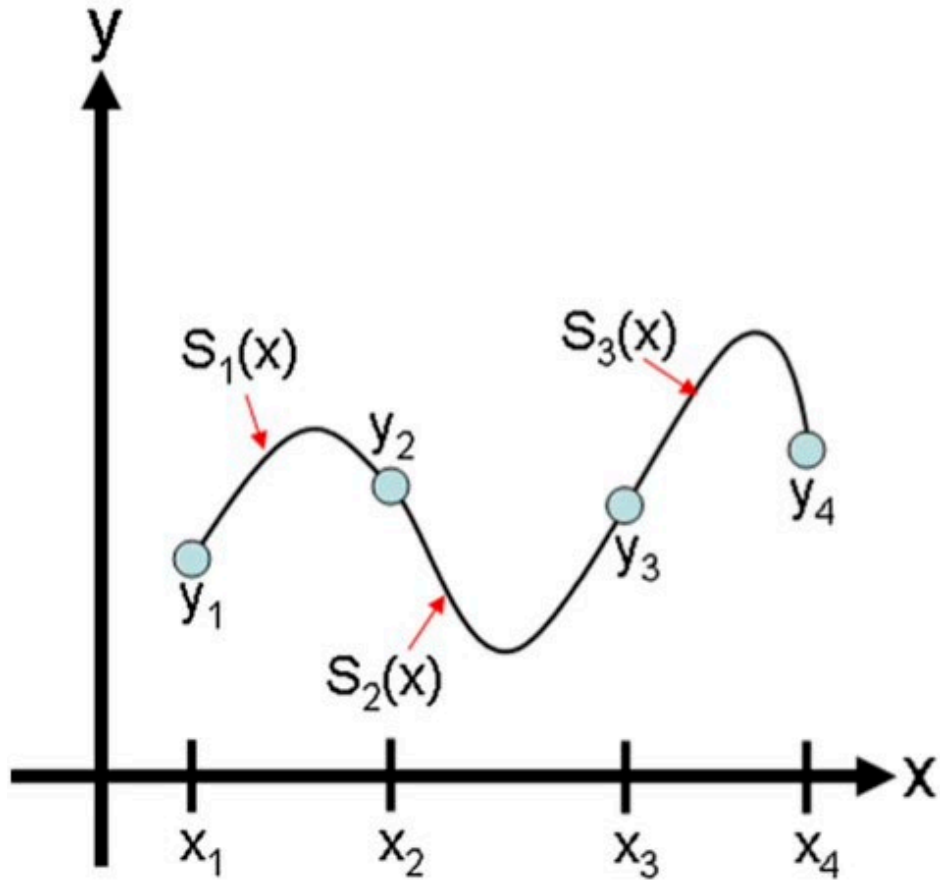
1.0

- Use **np.interp** for linear interpolation
- WARNING: make sure your data points are sorted in increasing order

Splines!



Piecewise Interpolation



- For N data points there is one unique $N-1$ degree polynomial that goes through all of them
- But high degree polynomials are terrible and oscillate like crazy
- Instead: define piecewise functions!
- We define a different function (i.e. a polynomial with different coefficients) between data points
- In this example there are 4 data points so we build 3 different polys = 12 unknown coefficients

Cubic Splines

We get lots of advantages skipping from linear to cubic!

Given N data points we construct $N-1$ cubic polynomials with **$4(N-1)$** unknown coefficients

Constraint	Number
Each $(N-1)$ poly goes through data points on both ends	$2(N-1)$
First derivatives match at interior points $P'_{i-1}(x_i) = P'_i(x_i)$	$N-2$
Second derivatives match at interior points $P''_{i-1}(x_i) = P''_i(x_i)$	$N-2$
TOTAL	$4N-6$

Need 2 more constraints! Use boundary conditions at endpoints

Cubic Splines

Common choices for constraints:

- **fixed-slope:** user specifies slope at endpoints
- **clamped:** first derivative at endpoints is zero
- **natural:** second derivative at endpoints is zero
- **not-a-knot** (*default*): if we require $P_1'''(x_2) = P_2'''(x_2)$ the third derivative matches at the first interior point then the first two segments P_1 and P_2 have the same coefficients. The “knots” are the data points, so now x_2 is no longer a true knot. Use this option if you don't have any information about the endpoint slopes

Cubic

<https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.CubicSpline.html#scipy.interpolate.CubicSpline>

scipy.interpolate. CubicSpline

```
class CubicSpline(x, y, axis=0, bc_type='not-a-knot',  
extrapolate=None)
```

[\[source\]](#)

Cubic spline data interpolator.

Interpolate data with a piecewise cubic polynomial which is twice continuously differentiable [1].

The result is represented as a `PPoly` instance with breakpoints matching the given data.

Parameters:

x : *array_like, shape (n,)*

1-D array containing values of the independent variable. Values must be real, finite and in strictly increasing order.

y : *array_like*

Array containing values of the dependent variable. It can have arbitrary number of dimensions, but the length along `axis` (see below) must match the length of `x`. Values must be finite.

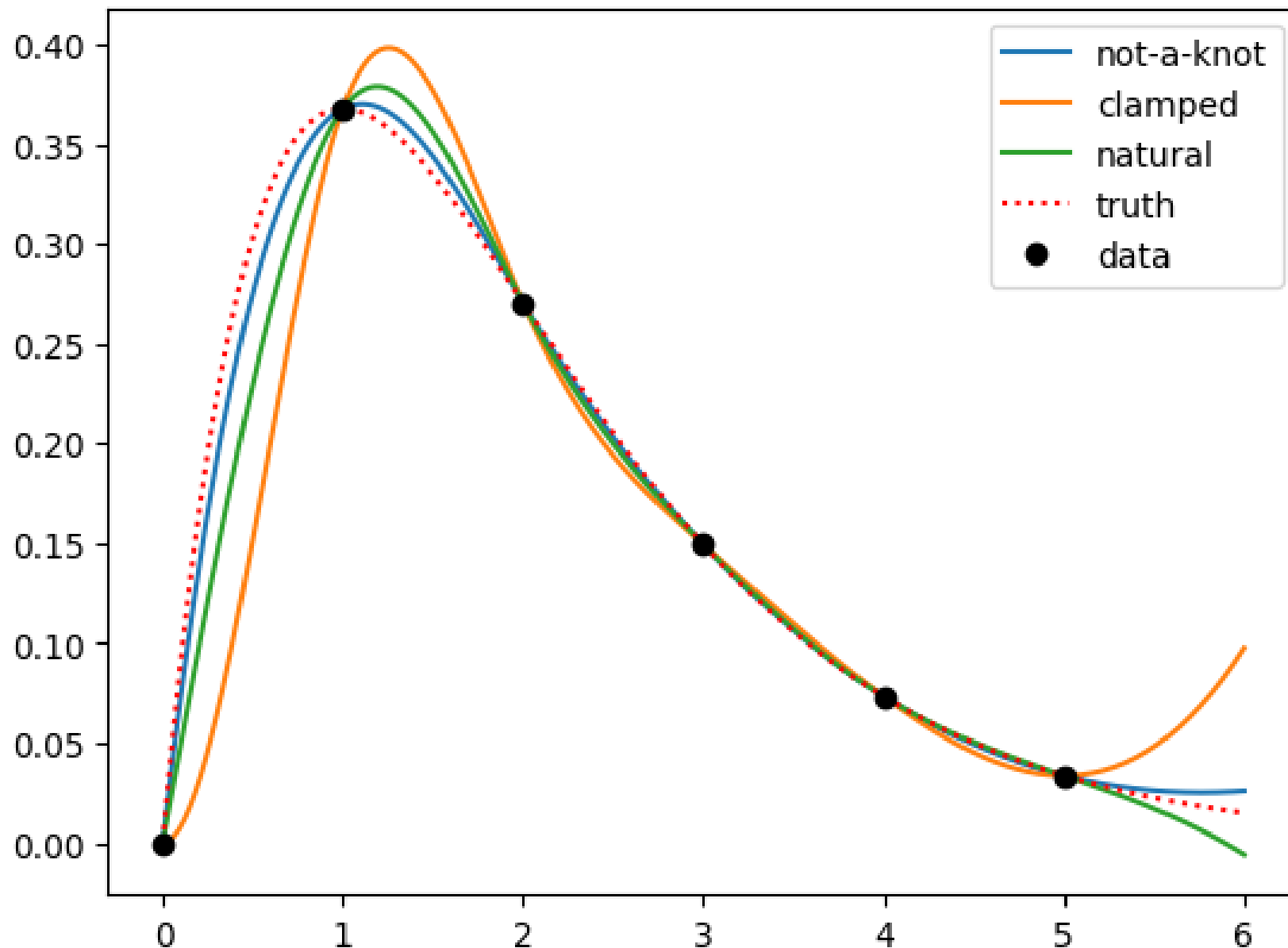
bc_type : *string or 2-tuple, optional*

Boundary condition type. Two additional equations, given by the boundary conditions, are required to determine all coefficients of polynomials on each segment [2].

If `bc_type` is a string, then the specified condition will be applied at both ends of a spline. Available conditions are:

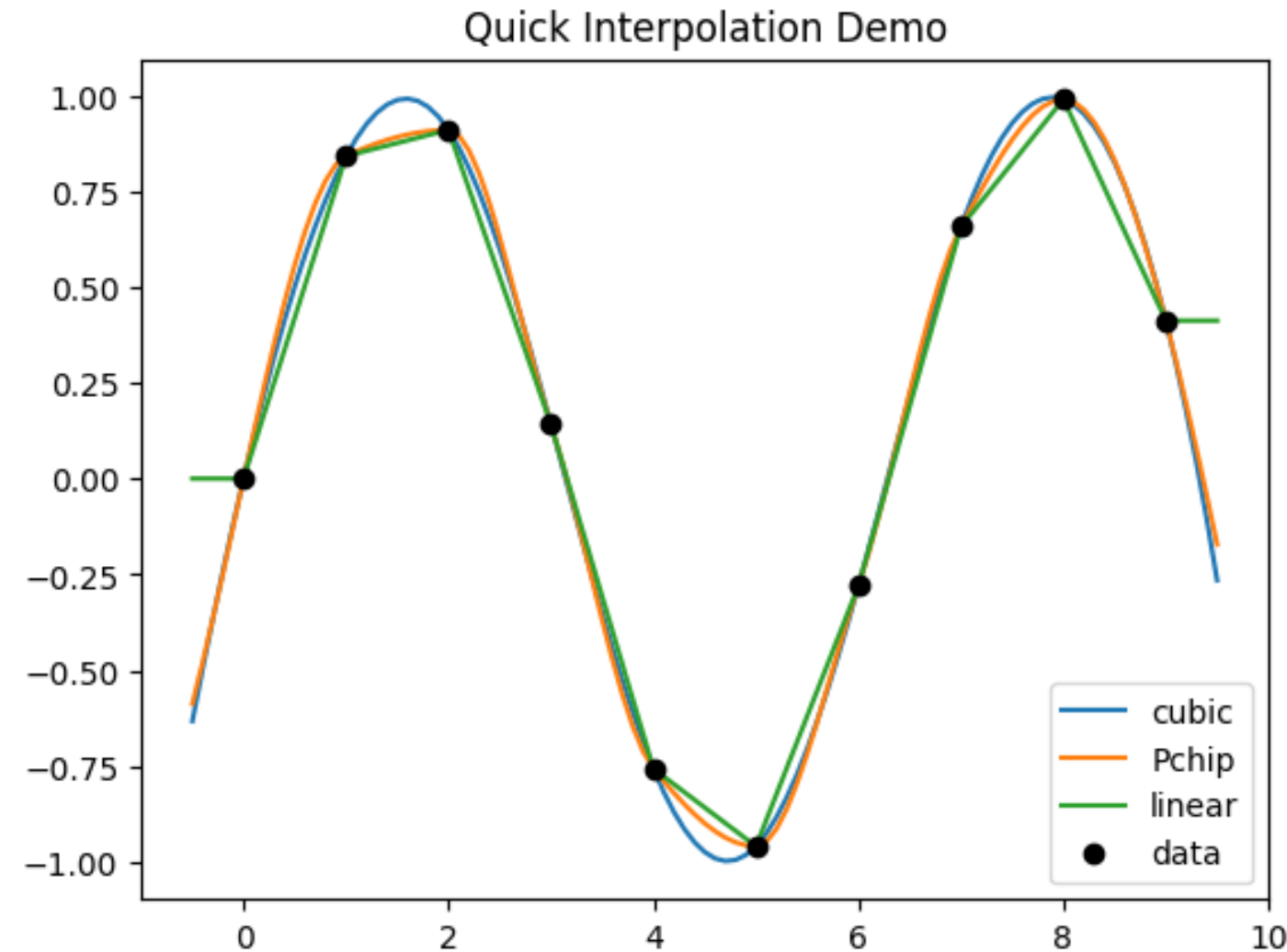
- 'not-a-knot' (default): The first and second segment at a curve end are the same polynomial. It is a good default when there is no information on boundary conditions.
- 'periodic': The interpolated functions is assumed to be periodic of period `x[-1] - x[0]`. The first and last value of `y` must be identical: `y[0] == y[-1]`. This boundary condition will result in `y'[0] == y'[-1]` and `y''[0] == y''[-1]`.
- 'clamped': The first derivative at curves ends are zero. Assuming a 1D `y`, `bc_type=((1, 0.0), (1, 0.0))` is the same condition.
- 'natural': The second derivative at curve ends are zero. Assuming a 1D `y`, `bc_type=((2, 0.0), (2, 0.0))` is the same condition.

Boundary Options



```
def fun1(x):  
    return x*np.exp(-x)  
  
xdata = np.arange(0,6)  
ydata = fun1(xdata)  
  
xs = np.linspace(0, 6, 200)  
  
for bc in ['not-a-knot', 'clamped', 'natural']:  
    cs = CubicSpline(xdata, ydata, bc_type=bc)  
    plt.plot(xs, cs(xs), label=bc)  
  
plt.plot(xs, fun1(xs), 'r:', label='truth')  
plt.plot(xdata, ydata, 'ko', label='data')  
plt.title('Boundary Options')  
plt.legend()  
plt.show()
```

The last option I'll mention is PCHIP (Piecewise Cubic Hermite Interpolating Polynomial). This is a middle ground between linear and cubic since the function and first derivatives match, but the second derivatives don't. The result are splines that are still smooth but much “flatter” with very little overshoot.



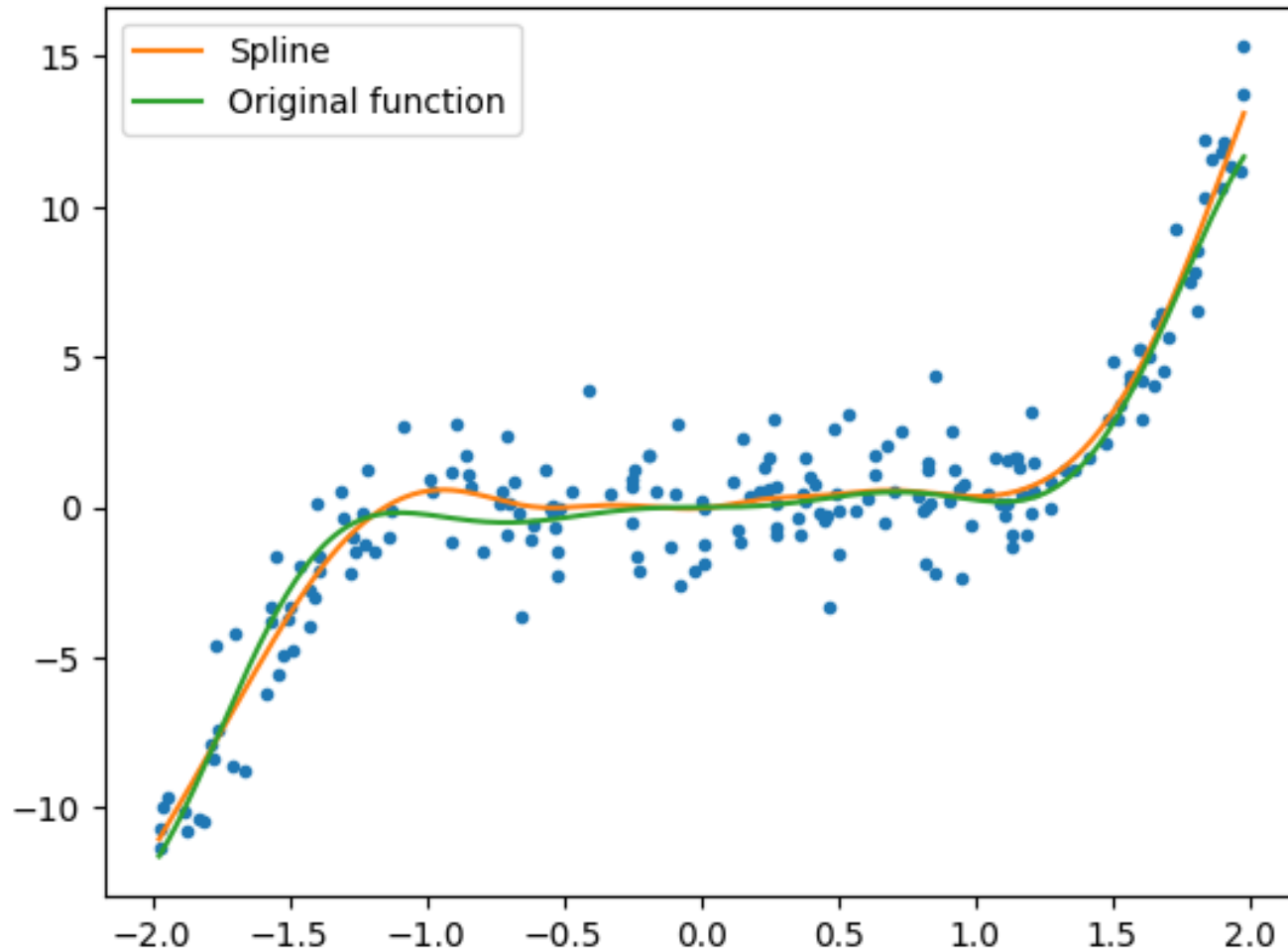
```
xdata = np.arange(10)
ydata = np.sin(xdata)
cs = CubicSpline(xdata, ydata)
csp = PchipInterpolator(xdata, ydata)

xs = np.arange(-0.5, 9.6, 0.1)

plt.plot(xs, cs(xs), label='cubic')
plt.plot(xs, csp(xs), label='Pchip')
plt.plot(xs, np.interp(xs, xdata, ydata), label='linear')
plt.plot(xdata, ydata, 'ko', label='data')
plt.title('Quick Interpolation Demo')
plt.legend()
plt.show()
```

Smoothing

There is a third option between fitting one function to every point vs playing connect-the-dots. If the data are noisy we can use a smoothing spline instead.



Interpolation Summary

- Lots of options: <https://docs.scipy.org/doc/scipy/tutorial/interpolate.html>
- **np.interp** for linear interpolation
- **scipy.interpolate.CubicSpline** for cubic
- 3rd option: if you really need an option that is smooth but as flat as possible you can use `scipy.interpolate.PchipInterpolator`
- WARNING: make sure your data points are sorted in increasing order

Things to Know

- When to interpolate and when to fit
- Fitting
 - polyfit and polyval
 - curve_fit
 - advanced tricks like brute or differential_evolution
- Interpolation
 - np.interp
 - scipy.interpolate.CubicSpline
 - be aware that other options exist like PCHIP and smoothing